ArgTune at AISA-ArabicFC: A Modular Pipeline for Arabic Function Calling

Sabah Baothman
Independent Researcher, Saudi Arabia
sabah.a.baothman@gmail.com

Abstract

We present ArgTune, a modular pipeline for Arabic function calling developed for the AISA-ArabicFC shared task. ArgTune separates tool selection from argument extraction using three LoRA-fine-tuned AISA-AR-FunctionCall-Think (270M) models: a tool-selection model, a general argument model, and a specialist model for seven challenging tools, allowing each model to focus on a more specific task. Targeted augmentation, data balancing, and query-grounded post-processing further address difficult argument cases. On the official test set, ArgTune achieved 0.9964 FnAcc, 0.8506 ArgEM, and an overall score of 0.9089, ranking 8th. The results highlight the remaining difficulty of exact argument extraction despite highly accurate tool selection.

1 Introduction

Function calling enables language models to interact with external tools by selecting a function and generating its arguments. This capability is central to agentic AI systems such as the broader AISA framework (Nacar et al., 2026c). In Arabic, function calling is particularly challenging due to dialectal and linguistic variation alongside strict structured-output requirements. The AISA-ArabicFC shared task evaluates these capabilities in Arabic (Nacar et al., 2026a).

Our experiments revealed a clear gap between tool selection and exact argument extraction. This motivated ArgTune, a modular pipeline that separates the two tasks and uses specialized argument extraction for difficult tools. Targeted augmentation, data balancing, and lightweight query-grounded post-processing were used to address recurring errors.

On the official test set, ArgTune achieved 0.9964 FnAcc, 0.8506 ArgEM, and an overall score of 0.9089, ranking 8th. The remaining gap between the two metrics identifies exact argument extraction as the main challenge.

Code and models are available at https://github.com/SabahBaothman/ArgTune.

2 Background

AISA-ArabicFC is a shared task on Arabic function calling for agentic AI systems. Given an Arabic user request and a set of available tools, the system must select the appropriate function and generate its arguments in a structured format (Nacar et al., 2026a). Table 1 shows an example from the development set.

Field	Example
Query	أريد تحويل 1500 ريال سعودي إلى دولار أمريكي.
Tool	convert_currency
Arguments	amount=1500, from=SAR, to=USD

Table 1: Example Arabic function-calling instance.

We participated in Track A (Core), which requires generating the function call (function name and its argument values) directly from the user request. The dataset contains 10,550 training and 545 development instances, covering five Arabic varieties: Modern Standard Arabic (MSA), Levantine, Egyptian, Gulf, and Maghrebi. Each request is paired with tool definitions and a structured target function call.

Our system builds on AISA-AR-FunctionCall-Think, the 270M-parameter model provided with the shared task. The model is based on Gemma 3 and was developed for Arabic structured tool calling using a data-centric fine-tuning approach (Nacar et al., 2026b). Prior work on function calling has shown that high-quality and diverse training data can improve reliability (Liu et al., 2025), while Arabic tool-calling studies report persistent performance challenges across different query forms (Kubrak et al., 2026). ArgTune uses a modular architecture to improve model focus by separating

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1597–1602
October 28-29, 2026 ©2026 Association for Computational Linguistics

tool selection from argument extraction and assigning seven challenging tools to a specialist argument model. To improve data quality and coverage, we also use targeted augmentation and balancing for difficult and underrepresented cases.

3 System Overview

ArgTune follows a modular architecture that decomposes Arabic function calling into distinct stages for tool selection and argument extraction. Given an input query, the system first applies preprocessing before passing the query to a dedicated tool-selection model. The predicted tool then determines which argument-extraction model is used: a specialist model handles seven challenging tools, while a general model covers the remaining tools. Finally, lightweight query-grounded post-processing rules are applied to improve the consistency of the extracted arguments and produce the final function call. Figure 1 illustrates the overall ArgTune workflow.

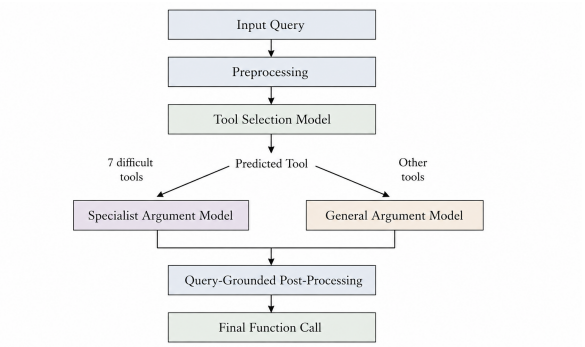


Figure 1: Overview of the ArgTune modular architecture.

3.1 Preprocessing

Before training, we reviewed the training examples and cleaned cases with inconsistent or noisy labels. Empty argument values were removed, and examples where the labeled arguments were not supported by the user query were either corrected when the intended value was clear or excluded from training.

Cleaning was performed separately for the argument models. For the general argument model, 50 original rows were excluded and four retained hotel examples had their guest-count labels corrected. For the specialist model, 133 rows were excluded: 6 from book_doctor_appointment, 32 from calculate_end_of_service, 85 from compare_prices, 5 from transfer_money, and 5 from calculate_customs. These removals mainly involved unsupported dates, ambiguous or inconsistent argument labels, and values not supported by the user query.

3.2 Tool Selection

The first stage of ArgTune is responsible for selecting the appropriate tool for each user query. We use a dedicated LoRA-fine-tuned AISA-AR-FunctionCall-Think model for this stage, allowing tool selection to be handled independently from argument extraction.

Most tools were already identified reliably by the base model. We therefore focused the additional training data on the remaining difficult cases rather than augmenting all tools equally. We added 30 targeted examples for translate_text and transfer_money, with 15 examples for each tool and variation across Arabic dialects. These examples were added once without oversampling, increasing the tool-selection training set from 10,550 to 10,580 examples. The selected tool is then used to route the query to the appropriate argument-extraction model.

3.3 Argument Extraction

After tool selection, ArgTune uses a separate model to extract the arguments required by the predicted tool. During our experiments, argument extraction was more challenging than tool selection, especially for tools with multiple or closely related argument fields. Instead of using a single argument model for all tools, we divided this stage into a general model and a specialist model.

The specialist model handles seven tools that showed more difficult argument-extraction patterns: order_food, book_doctor_appointment, check_insurance_coverage, compare_prices, calculate_customs, transfer_money, and calculate_end_of_service. The general model handles the remaining tools. Both models are LoRA-fine-tuned from AISA-AR-FunctionCall-Think and receive the user query together with the tool selected in the previous stage.

Training for both argument models was supported by targeted augmentation and selective oversampling. For the general model, we added 264 unique examples: 151 for search_hotels, 22 for get_weather, 12 for search_medications, 3 for search_quran, 36 for calculate_zakat, and 40 for convert_currency. The first 188 targeted examples were used three times each, while the re-

maining 76 examples were added once. We also duplicated 37 search_quran training examples to improve the balance of the search_type argument. This produced 7,441 training examples, including 6,764 cleaned original examples and 677 augmented or oversampled examples.

For the specialist model, we added 52 unique targeted examples: 26 for transfer_money, 6 each for calculate_end_of_service, calculate_customs, and check_insurance_coverage, 5 for compare_prices, and 3 for book_doctor_appointment. No new examples were added for order_food. After cleaning, the 3,553 original specialist examples were combined with the targeted examples and balanced across tools. Tools were initially oversampled toward 622 examples, with at most three occurrences of each source row. End-of-service termination values were further balanced, and selected targeted examples received between four and nine total training exposures. The final specialist training set contained 4,707 examples.

Model	Unique Aug.	Added Exp.	Final Train
Tool selection	30	30	10,580
General args.	264	677	7,441
Specialist args.	52	1,154	4,707

Table 2: Training augmentation and oversampling by model. In total, 346 unique targeted examples were added.

3.4 Query-Grounded Post-processing

After argument extraction, we apply a fixed set of lightweight, tool-specific post-processing rules. The rules use explicit evidence from the user query to remove unsupported arguments, recover explicitly stated values, and normalize recurring formatting inconsistencies. They modify only the predicted arguments and do not change the selected tool. Table 3 lists the complete set of rules used in the final pipeline. A final schema-level cleanup removes empty values and arguments that are not valid for the selected tool. On the development set, the rules modified 24 of the 545 predictions.

Tool	Post-processing rule
order_food	Remove the Arabic "restaurant" prefix from restaurant names.
calculate_customs	Remove the Arabic "device" prefix from product categories.
calculate_end_of_service	Map explicit termination cues to the corresponding schema value; remove ambiguous values.
transfer_money	Normalize IBANs by removing spaces and trailing .0.
calculate_zakat	Remove invalid weight-unit currencies and normalize crop to crops.
search_umrah_packages	Remove departure_date when no date is stated.
translate_text	Recover quoted text, clean quotation artifacts, and normalize the target language.
check_visa_status	Remove visa_number when no numeric identifier is stated.
search_hotels	Recover explicit guests and dates; remove unsupported guest or date arguments.
get_weather	Normalize days from explicit temporal expressions and remove invalid values.
get_qibla_direction	Remove the Arabic "city" prefix from city names.
convert_currency	Recover amounts from supported Arabic written-number expressions.
All tools	Remove empty values and arguments outside the selected tool schema.

Table 3: Query-grounded post-processing rules used in the final pipeline.

4 Experimental Setup

We used the training and development splits provided by the AISA-ArabicFC shared task (Nacar et al., 2026a). The training split was used for fine-tuning, the development split for model development and error analysis, and the blind test set was reserved for official evaluation. Training data were cleaned, augmented, and selectively oversampled as described in the System Overview. No external training data or generation APIs were used.

All three models were initialized from AISA-AR-FunctionCall-Think (270M) and fine-tuned with LoRA for one epoch using a batch size of 1. The tool-selection model used rank 8 with a learning rate of 2×10^{-5}; the general argument model used rank 8 with 1×10^{-4}; and the specialist model used rank 16 with 1×10^{-4}. Experiments were conducted on NVIDIA T4 GPUs using PyTorch 2.10.0, Transformers 5.15.0, Datasets 5.0.1, Accelerate 1.14.0, and PEFT 0.20.0; library URLs are provided in Appendix A.

Evaluation follows the official Track A metrics. FnAcc measures whether the correct function is selected, while ArgEM requires an exact match between the predicted and gold argument sets.

For development reporting, ArgEM is computed over the 500 positive function-calling examples only; the 45 negative examples are excluded from the ArgEM denominator.

The overall score combines both metrics as

$0.4\,\text{FnAcc} + 0.6\,\text{ArgEM}$.

5 Results

ArgTune achieved an overall score of 90.89% on the official test set, with FnAcc of 99.64% and ArgEM of 85.06%, ranking 8th in the shared task. On the development set, FnAcc reached 100.00% while ArgEM was 81.40%, resulting in an overall score of 88.84%. After correction, development ArgEM was lower than the blind test ArgEM by 3.66 percentage points, despite using development observations to guide augmentation and post-processing. Table 4 summarizes the development and official test results.

Split	FnAcc	ArgEM	Overall
Dev	100.00	81.40	88.84
Test	99.64	85.06	90.89

Table 4: ArgTune results on the development and official test sets (%).

5.1 Component Analysis

We analyze the main development steps that led from the initial single-model configuration to the final ArgTune pipeline. All comparisons in this section are based on the development split.

Single Model: Our initial configuration used a single model to predict the complete function call. It achieved 98.17% FnAcc but only 46.60% ArgEM, resulting in an overall score of 67.23%. The large gap showed that most remaining errors came from argument extraction rather than tool selection.

Task Separation: Based on this observation, we separated tool selection from argument extraction and trained a dedicated model for each task. Allowing the argument model to focus only on extracting argument values increased ArgEM from 46.60% to 52.60% on the development set. However, further tuning produced limited and inconsistent gains across individual tools.

Argument Specialization: Error analysis showed that argument mistakes were concentrated in seven tools. We also observed that changes intended to improve these tools could improve some while degrading others. We therefore separated argument extraction into a general model for the remaining tools and a specialist model for these seven difficult tools. This specialization increased ArgEM to 72.00% and formed the final three-model architecture of ArgTune. The large gain indicates that isolating the difficult tools allowed the specialist model to focus on their more challenging argument patterns without interfering with the remaining tools.

Targeted Augmentation: After establishing the three-model pipeline, targeted augmentation was applied to the remaining difficult cases rather than increasing the data uniformly. Additional examples and selective oversampling were introduced separately for tool selection and argument extraction. With targeted augmentation, the three-model pipeline reached 100.00% FnAcc and 79.60% ArgEM on the development set.

Query-Grounded Post-Processing: Finally, lightweight rules were applied to recurring argument errors that could be resolved directly from the user query. The rules mainly removed unsupported values and recovered or normalized explicitly stated arguments. Post-processing changed ArgEM from 79.60% to 81.40%, producing the final development score reported in Table 5.

Configuration	FnAcc	ArgEM	Overall
Single model	98.17	46.60	67.23
Task separation	98.17	52.60	70.83
Argument specialization	98.17	72.00	82.47
+ Targeted augmentation	100.00	79.60	87.76
+ Post-processing	100.00	81.40	88.84

Table 5: ArgTune development performance across system stages (%).

5.2 Error Analysis

Error analysis was performed on the full development split, which contains 545 examples: 500 positive function-calling examples and 45 negative examples. Among the 500 positive examples, 93 remained incorrect after the final query-grounded post-processing stage. In all 93 cases, the predicted tool was correct, indicating that the remaining errors were entirely due to argument extraction. We identified three main error types: incorrect argument values, missing arguments, and extra unsupported arguments. Table 6 summarizes their percentages and provides an example of each type.

Error Type	Count	%	Example
Wrong value	81	87.1	AED predicted as MAD.
Missing argument	10	10.8	days=1 missing from a "weather" query.
Extra argument	4	4.3	destination_country added to a customs query.

Table 6: Error types among the 93 remaining incorrect positive development examples; an example may contain multiple error types.

We further examined how these errors were distributed across tools. Table 7 shows the tools accounting for the largest shares of the 93 residual errors.

Tool	Errors	% of Errors
book_doctor_appointment	13	14.0
compare_prices	13	14.0
transfer_money	11	11.8
order_food	9	9.7
search_medications	7	7.5
calculate_customs	7	7.5

Table 7: Tools with the largest shares of the 93 residual argument errors on the positive development set.

Although specialization improved ArgEM, most of the tools with the largest remaining error counts were still specialist tools. This suggests that these tools remain particularly difficult for exact argument extraction.

Overall, the analysis shows that the main remaining challenge is not selecting the correct tool, but extracting the exact argument values required by the reference. Future improvements should therefore focus on more precise value extraction and schema-consistent argument generation.

5.3 Dialect Robustness

We additionally analyze ArgTune using the Track C dialect-robustness diagnostics provided by the shared-task organizers. Track C macro-averages the per-dialect Overall scores across MSA, Gulf, Egyptian, Levantine, and Maghrebi with equal weight. Table 8 reports ArgTune's per-dialect performance. The Maghrebi score of 0.735 and the resulting 0.184 dialect gap are based on only 11 test examples. Although this was the widest dialect gap among the top ten systems, the small sample size means that the result should be interpreted with caution.

Macro	MSA	Gulf	Egy	Lev	Maghrebi	Gap
0.8754	0.920	0.918	0.918	0.886	0.735	0.184

Table 8: ArgTune Track C dialect-robustness results.

Performance was similar across MSA, Gulf, and Egyptian, but lower for Maghrebi, resulting in a dialect gap of 0.184.

6 Conclusion

We presented ArgTune, a modular pipeline for Arabic function calling that separates tool selection from argument extraction and uses specialized modeling for difficult tools. ArgTune achieved an official test score of 0.9089, ranking 8th, with highly accurate tool selection while exact argument extraction remained the main challenge. Targeted augmentation and query-grounded post-processing further improved difficult cases. Future work will focus on improving argument extraction while reducing reliance on manually defined post-processing rules.

Limitations

While ArgTune achieved strong results, there remain several opportunities for further improvement. The targeted augmentation and query-grounded post-processing were designed around recurring patterns observed during development, and future work could explore broader augmentation strategies to improve coverage of less common query forms. In addition, although specialization substantially improved argument extraction, some of the more challenging specialist tools still show room for further accuracy gains. Finally, the Maghrebi test subset contains only 11 examples, so additional evaluation on a larger sample would provide a more reliable assessment of performance for this dialect.

Acknowledgments

We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, the baseline model, and the evaluation framework for this shared task.

A Implementation Details

PyTorch: https://pytorch.org/
Transformers: https://huggingface.co/docs/transformers/
Datasets: https://huggingface.co/docs/datasets/
Accelerate: https://huggingface.co/docs/accelerate/
PEFT: https://huggingface.co/docs/peft/

References

Konstantin Kubrak, Ahmed El-Moselhy, Ammar Alsulami, Remaz Altuwaim, Hassan Ismail Fawaz, and Faisal Alsaby. 2026. Arabic prompts with english tools: A benchmark. *arXiv preprint arXiv:2601.05101*.

Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, et al. 2025. Toolace: Winning the points of llm function calling. In *International Conference on Learning Representations*.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic ai systems.